dry-Film-lab

Architecture & Technical Specification | PC build and source audit | 2026

1. Executive summary

dry-Film-lab is a Java camera-app codebase that stages Sony camera look parameters, presents a recipe browser over the live camera surface, and includes a reflective movie-control path. The requested target is the Sony NEX-5R / CX68000 on its PlayMemories-era Android API 10 platform. This document describes the checked source snapshot and locally built Java-only package; it does not claim a hardware-validated camera release.

The ?S-Log Style? recipe is a deliberately flat Creative Style approximation: Neutral, saturation ?2, contrast ?3, sharpness ?1, DRO level 5. It does not implement Sony S-Log gamma, a transfer function, or a color-space transform. A real log curve would require a supported camera picture-profile pipeline or pixel processing; neither is present in this snapshot.

Attribution and provenance

The UI watermark in this tree reads ?based on FilmLab by Marten Boet ? mod: Adriel Haryono.? The repository README and LICENSE separately identify Andr? Domingues (voxivoid) as author/copyright holder for the Recipe Lab source. These are distinct provenance statements in the checked files; this document preserves the requested watermark attribution without asserting an unverified source lineage or replacing the repository license.

Target constraints and build status

Item	Checked source / build facts
Runtime	Android minSdk 10, targetSdk 10; landscape activity; Sony proprietary CameraEx access uses reflection.
Processing model	Preview is delivered to a SurfaceView; no app-owned preview-frame callback or native pixel pipeline was found.
Native store	No JNI settings implementation in this snapshot; NativeBackup is an in-memory HashMap mock.
Artifact	dry-Film-lab.apk built and v1 signature verified locally; camera operation and persistent settings are not verified.

Use this document as an engineering handoff for the present snapshot, not as a guarantee of firmware compatibility or production readiness.

2. Software architecture & data flow

The application separates UI/event handling, parameter translation, camera preview control, and persistence-facing calls. The first three are visible in Java. The persistence boundary is currently represented by a mock and must be restored before on-camera settings can be considered durable.

Component map

Component	Role and boundary
MainActivity	Owns Activity/UI lifecycle, active and staged recipe state, Sony key events, Camera/CameraEx references, preview parameter application, capture callback, and REC indicator.
Recipes / Library	Recipe catalog and browsing. S-Log Style is placed in Sony group; recipe values are plain Java data.
Params	Maps recipe values to live preview parameter strings and settings-store byte writes, including mirrored DRO, white-balance, exposure, and quality slots.
CameraEx / ISP	Proprietary Sony camera interface opened reflectively. Runtime preview parameters are applied through Camera.Parameters; movie controls call CameraEx methods reflectively.
NativeBackup	In this folder: Java HashMap mock implementing read/write/attr/sync. It is not native, durable, or connected to camera firmware.
VideoRecorder	Reflective movie start/stop wrapper, UI listener dispatch, and best-effort background AvindexStore notification.

Data flow (intended camera-backed behavior)

Flow	Description
Selection	Sony physical key scan event ? MainActivity stages a recipe from Recipes/Library.
Live preview	Params.preview(...) ? Camera.Parameters values ? camera.setParameters(...) ? Sony ISP produces the SurfaceView image.
Persistent store	Params.writes(...) ? NativeBackup.read/write/attr/sync boundary. In a complete native port this boundary must invoke the camera settings-store implementation; this snapshot only mutates process memory.
Capture / video	Hardware capture or CameraEx movie recording encodes media; Java handles still-file output and issues best-effort media indexing after stop/write.

State and threading

MainActivity owns current/staged parameter arrays and commits changes through a defined write list. Preview updates are event-driven on selection/adjustment rather than a Java frame loop. The REC indicator is scheduled at 500 ms intervals. Still-image bytes arrive from the camera callback and are written to storage there; AvindexStore notification is moved to a low-priority worker thread. UI redraw and callback allocation have not been measured on the NEX-5R.

Critical deployment gap: the mock makes unit-level mapping tests possible, but a successful Java compile cannot establish that the camera stores settings or that the proprietary calls exist on the target firmware.

3. ISP offload & ?S-Log Style?

The practical design choice in this snapshot is to ask the camera ISP to render and encode the image rather than transform each preview frame in app code. This avoids introducing a CPU pixel loop on a memory-constrained API 10 camera and leaves video/photo encoding in the firmware pipeline. It is not equivalent to implementing an NDK effect.

Preset values

Control	Recipe value	Intent
Creative Style	Neutral (stored style enum 3)	Use the least strongly stylized base mode available in this mapping.
Saturation	?2	Reduce color intensity while retaining color.
Contrast	?3	Flatten the in-camera contrast setting.
Sharpness	?1	Slightly soften edge enhancement.
DRO	Level 5 (maximum manual level)	Request the highest mapped DRO level for shadow lift; actual behavior is firmware-controlled.

Parameter path

The Sony recipe object encodes these values in Recipes.java. MainActivity stages them and Params translates them into preview parameters and settings-store writes. DRO uses two store slots: the main mode byte maps level 5 to value 6, and the level byte maps level 5 to 6. When a real NativeBackup implementation is present, both live and stored state must agree; the current Java mock does not meet that persistence requirement.

What it does not do

?S-Log Style? is a creative preset name, not a log profile. No gamma curve, matrix, per-pixel operation, JNI source, or NDK-built shared object is in this snapshot. The camera encodes using its own ISP and firmware behavior. Neutral/contrast/saturation/DRO cannot be claimed to preserve highlight latitude like S-Log; DRO can affect tone rendering and is not a substitute for a log transfer curve.

Memory/performance implications

Source inspection found no Camera.PreviewCallback or app-side frame-processing loop. Therefore a claim of zero allocation in preview/onDraw is not supported as a measured result. The application does allocate normal UI/event objects and receives a full-resolution JPEG byte array in the capture callback. The callback now closes its file stream in a finally block, removes partial output on failure, and avoids reusing an existing DSC filename. No device-level dropped-frame or Dalvik GC benchmark was available.

Treat performance statements as architectural intent only until measured on the actual NEX-5R with sustained photo/video capture and logcat instrumentation.

4. Video pipeline & media indexing

Recording is delegated to Sony firmware through reflective CameraEx method calls; the Java layer controls state, presents a status badge, then requests a media index update after stop. Exact signatures and media-store behavior depend on the installed camera firmware.

Control path

Step	Implementation observed
Key event	Keys.java defines Sony ScalarInput scan code K_MOVIE = 515. MainActivity routes that scan code to VideoRecorder.toggle(). This is not Android KEYCODE_CAMERA 130; they are different code spaces.
Start	VideoRecorder reflectively resolves and invokes CameraEx.startMovieRec(); exceptions are converted to a listener error and recording state is reset.
Stop	stopMovieRec() is invoked reflectively. State is cleared in finally; stop exceptions are swallowed, so a false UI state does not prove the firmware finalized a valid file.
Indicator	MainActivity updates the REC overlay and runs a 500 ms status/blink tick while recording.
Index	A background thread first tries CameraEx.getLastMovieFile(); if absent, scans DCIM/100MSDCF for the newest MP4/MTS/MOV, then reflectively calls AvindexStore.notifyNewFile(path). Still photos use the same best-effort notifier after the JPEG stream closes.

Important uncertainties

getLastMovieFile() and AvindexStore.notifyNewFile(String) are reflective method-name assumptions in this code, not validated NEX-5R APIs. The fallback chooses the newest matching file and may race delayed file finalization or select a prior recording. AvindexStore failure is logged, but cannot repair a recording that the camera has not flushed. No camera, SD-card, or PMCA transport was available for a hardware test.

Operational test on target hardware

On a disposable SD card, verify one short recording, wait for firmware finalization, confirm the resulting file is playable and visible in the camera gallery, then repeat with the movie button and app lifecycle. Confirm the scan code in the app key logger before relying on the binding. Check logs for start/stop reflection failures and AvindexStore warnings. Do not infer file integrity from the REC badge alone.

The keypad code requested in the original task (?KEYCODE_CAMERA = 130?) is not what the source binds. The source uses Sony ScalarInput scan code 515; retain that distinction in any deployment guide.

5. Testing, bug audit & deployment

Local checks cover pure Java mappings and Android package construction. They do not replace device validation of camera-private APIs.

Test and audit results

Check	Result / limitation
JUnit	310 camera-free Java tests passed in the prior full-suite run. The runner covers Recipes, Params, key handling, recipe formats/library, localization, and developer tools; MainActivity/CameraEx are excluded because they require Android/camera runtime.
Build	Windows build.cmd produced dry-Film-lab.apk with API 28/build-tools 30.0.3, Java 17 source/target 8, D8 min API 10, zipalign, and v1 signature verification. This is a Java-only package build, not an NDK build.
Resolved audit issue	Still-image output now closes its stream on both success and failure, removes incomplete files after save errors, and skips occupied DSC names to avoid timestamp-derived overwrites.
Known blocker	NativeBackup is only an in-memory mock; recipe persistence to Sony camera settings is absent. NEX-5R operation and reflective video/media calls remain unverified.
Performance caveat	No preview callback/frame transformation loop was found. No measured ?zero GC? or zero-frame-drop claim is justified; still JPEG callback and UI objects allocate.

Build locally (PowerShell)

Set JAVA_HOME=D:\jdk-17, ANDROID_SDK=D:\android, BUILD_TOOLS=30.0.3, and PLATFORM_JAR=D:\android\platforms\android-28\android.jar, then run .\build.cmd from the project root. Output: dry-Film-lab.apk. The defaults in build.cmd may differ by workstation; override paths explicitly.

PMCA install and live log workflow

For a device-ready build only, after restoring the native settings-store implementation and validating target compatibility, the project contributor guide uses: pmca-console.py install -d native -f RecipeLab.apk. The checked debug helper monitors logs with: pmca-cli.exe logcat -v time DalvikVM:D AndroidRuntime:E CameraEx:V *:S. Tool availability, USB pairing, install success, and on-camera behavior were not verified here.

Release gate

Do not deploy this Java-only APK as a functional camera settings app. First restore the correct licensed JNI backup/store implementation, build with the native pipeline, confirm durable recipe writes after a power cycle, verify the Sony method signatures and movie path on the NEX-5R, and run sustained capture tests on the device.